

操作系统原理第六章作业

姓名：林宏宇 学号：23320093 截止日期：2025 年 04 月 28 日

完成日期：2023 年 04 月 17 日

Question 1:

6.1 第一个著名的正确解决两个进程的临界区问题的软件方法是由 Dekker 设计的。两个进程 P_0 和

习 题 • 199 •

P_1 共享以下变量：

```
boolean flag[2]; /*initially false*/  
int turn;
```

进程 P_i ($i=0$ 或 1) 的结构见图 6.25，另一个进程为 P_j ($j=0$ 或 1)。试证明这个算法满足临界区问题的所有三个要求。

```
do {  
    flag[i] = TRUE;  
    while (flag[j]) {  
        if (turn == j) {  
            flag[i] = false;  
            while (turn == j)  
                ; // do nothing  
            flag[i] = TRUE;  
        }  
    }  
    // critical section  
    turn = j;  
    flag[i] = FALSE;  
    // remainder section  
} while (TRUE);
```

图 6.25 Dekker 算法中的进程 P_i 结构

Answer 1:

理论作业5:第6章习题

1. 临界区问题的三个要求: ①互斥 ②进步 ③有限等待

①互斥: P_i 在临界区执行, 则 P_j 不能在临界区中执行

i) $flag[i] = true$; 表示 i 状态想进入临界区.

ii) $turn = j$; 说明 j 应在 i 之前排队等待, P_j 将先进入临界区, P_i 无法进入

②进步: 如果临界区无进程, 择一而入从等待中挑选进入临界区, 且不会无限推迟.

i) $turn = j$ 可保证下一次进入的是 j , 其它无法插队 $flag[i] = false$.

ii) 只有当 j 进程结束时 $flag[i] = true$, 才有机会成为下一个被挑选的进程

③有限等待: 从发出请求到允许进入临界区, 其它进程不能无限进入

i) 第一次 $flag[i] = True$ 发出请求, 但进程在运行, $flag[i] = false$.

ii) $turn = j$ 不成立后, $flag[i] = true$ 又可申请进入临界区

iii) 每一轮都进行上诉操作, 每个进程都有机会进入临界区, 且不会无限等待.

Question 2: 术语忙等的含义是什么? 操作系统中其他类型的等待有哪些? 忙等能否完全避免? 为什么?

Answer 2:

2. ①忙等待: 当进程在等待某事件发生时, 持续性地判断条件而执行空语句. 用来持续等待某个条件使其满足某一状态, 如不满足则一直进行浪费 CPU

②其它类型: 阻塞等待、休眠等待、事件等待.

③能否完全避免: 不能

④为什么: 当临界区有空位时需要快速地从等待的进程中挑选一个可以进入临界区, 为了保证快速、高效地响应, 则忙等仍有需应用.

Question 3: 试说明如果 wait 和 signal 操作不是原子化操作, 那么互斥可能是不稳定的。

Answer 3:

3. 说明如果 wait 和 signal 不是原子化操作, 那么互斥可能不稳定。

如果非原子化:

① wait: 在执行 wait 操作时, 如果其它进程调用了 signal, 那么将导致 wait 值修改, 进程将被错误地阻塞或唤醒。

② signal: 在执行 signal 时如果其它进程调用了 wait, 那么将导致 signal 被修改, 可能导致进程多被唤醒, 造成死锁问题或数据竞争问题。

③ 原子化可保证 wait, signal 执行时不被中断, 完成操作后其它进程才能介入, 保证互斥性和稳定性。

KOKUYO